

Trabalho 1: Mundo dos Blocos de Tamanho Variável via SAT Solver

Fundamentos de Inteligência Artificial — Prof. Edjard Mota

Equipe Livia_Lorenzo — Livia Karolina Gomes do Nascimento, Lorenzo Augusto da Costa Freitas

Outubro de 2026

Sumário

1	Introdução ao Problema	1
2	Descrição Formal do Mundo dos Blocos de Tamanho Variado	1
2.1	Blocos, mesa, pontos e slots	1
2.2	Predicados	1
2.3	Regra de estabilidade	2
2.4	Ação $move(b, y, p, t)$: pré-condições e efeitos	2
2.5	Ajuste em relação ao manual	2
2.6	Ordem parcial	2
3	Codificação CNF	2
3.1	Variáveis proposicionais	2
3.2	Grupos de cláusulas	3
4	Exemplos dos Três Cenários Codificados	3
5	Mapeamento: Descrição Formal $\rightarrow$ Código	4
6	Execução Passo a Passo	4
6.1	Planos encontrados	5
6.2	Execução manual: Situação 1, $S_0 \rightarrow S_{f4}$	5
7	Interpretação da Saída do SAT Solver	5
7.1	O miniSAT não conhece nomes simbólicos	5
7.2	Da saída numérica ao plano em português	5
7.3	Derivação de <i>on</i>	5
7.4	Verificação independente	5
7.5	Regra de ouro	6

1 Introdução ao Problema

No Mundo dos Blocos clássico todos os blocos têm o mesmo tamanho. Aqui os blocos têm comprimentos diferentes, ficam em qualquer posição horizontal de uma mesa com espaços vazios, podem ficar em ponte sobre dois blocos e só permanecem de pé se tiverem apoio suficiente. O objetivo é gerar um plano (uma sequência de ações) que leva um estado inicial a um estado meta, descrevendo o domínio em lógica de primeira ordem (LPO), traduzindo para lógica proposicional (CNF) e resolvendo com o miniSAT.

A ação de movimento é *qualitativa*: em vez de “mover para o nível 2, posição 3”, dizemos “mover o bloco d para cima do bloco c , começando no ponto 0”.

2 Descrição Formal do Mundo dos Blocos de Tamanho Variado

2.1 Blocos, mesa, pontos e slots

$B = \{a, b, c, d\}$ com comprimentos $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$, $\ell(d) = 3$. A mesa T tem os pontos $X = \{0, \dots, 6\}$ e os *slots* $s_i = [i, i+1]$, $i \in \{0, \dots, 5\}$: são **6 slots**, não 7. Um bloco b que começa no ponto p cobre os slots $p, \dots, p + \ell(b) - 1$, e as posições válidas são $p \in \{0, \dots, 6 - \ell(b)\}$. Os níveis vão de 0 (mesa) a 3.

2.2 Predicados

Predicado	Tipo	Significado
$at(b, p, t)$	estado	b começa no ponto p em t
$lev(b, l, t)$	estado	b está no nível l em t
$clr(b, t)$	estado	nada está sobre b em t
$on(b, y, t)$	derivado	b apoiado em $y \in B \cup \{T\}$
$stable(b, p, l, t)$	derivado	apoio suficiente
$move(b, y, p, t)$	ação	mover b para cima de y , começando em p

A relação on é derivada, não codificada:

$$on(b, y, t) \leftrightarrow \exists p, p_y, l. at(b, p, t) \wedge at(y, p_y, t) \wedge lev(b, l, t) \wedge lev(y, l - 1, t) \wedge overlap(b, p, y, p_y).$$

Um bloco pode ter vários apoios (ponte): $on(d, a, t) \wedge on(d, b, t)$.

2.3 Regra de estabilidade

$$stable(b, p, l, t) \leftrightarrow \#\{s \in [p, p + \ell(b) - 1] : \exists b' \neq b, cov(b', s, l - 1, t)\} \geq \lceil \ell(b)/2 \rceil.$$

Assim d pode ficar em ponte sobre dois blocos de comprimento 1 separados por um vazio, mas não pode ficar apoiado em um só slot.

2.4 Ação $move(b, y, p, t)$: pré-condições e efeitos

Pré-condições: (1) $clr(b, t)$; (2) se y é bloco, o span de b em p sobrepõe o de y ; (3) o destino difere da posição atual; (4) os slots ocupados por b no nível-alvo estão livres; (5) $stable(b, p, \text{nível-alvo})$; (6) nível-alvo ≤ 3 .

Efeitos (adds e deletes):

Elemento	Add	Delete
at	$at(b, p)$	$at(b, p_{\text{antigo}})$
lev	$lev(b, lev(y) + 1)$ (ou $lev(b, 0)$ se $y = T$)	$lev(b, l_{\text{antigo}})$
clr	$clr(z)$ para o bloco z que ficou sem nada em cima	$clr(y)$, se y é bloco
on (derivado)	apoios novos de b	apoios antigos de b

Todos os demais blocos permanecem inalterados (frame axioms).

2.5 Ajuste em relação ao manual

O manual exige $clr(y, t)$, o topo de y *inteiramente* livre. Testamos essa regra: com ela, os estados S_{f1} , S_{f2} (Situação 1), S_5 (Situação 2) e S_7 (Situação 3) ficam inalcançáveis, pois neles dois blocos ficam lado a lado sobre o mesmo bloco e o segundo nunca consegue subir. O miniSAT respondeu UNSATISFIABLE para todo $T \leq 12$. A pré-condição foi trocada por “os slots do topo de y que b ocupará estão livres”, que é a pré-condição (4).

2.6 Ordem parcial

Escrevemos $\varphi_1 \prec_P \varphi_2$ para “ φ_1 deve ter sido atingida antes (ou junto) de φ_2 valer”. Cada φ é um par (bloco, ponto, nível). Para todo t :

$$\neg\varphi_2(t) \vee \varphi_1(0) \vee \varphi_1(1) \vee \dots \vee \varphi_1(t).$$

3 Codificação CNF

3.1 Variáveis proposicionais

Variável	Índices
$at(b, p, t)$	$b \in B, p \in [0, 6 - \ell(b)], t \in [0, T]$
$lev(b, l, t)$	$b \in B, l \in [0, 3], t \in [0, T]$
$clr(b, t)$	$b \in B, t \in [0, T]$
$move(b, y, p, t)$	$y \in B \cup \{T\}, y \neq b, t \in [0, T - 1]$
$acima(x, y, t)$	auxiliar: x imediatamente acima de y
$ord_k(b, p, l, t)$	auxiliar da ordem parcial

A relação *on não* vira variável.

3.2 Grupos de cláusulas

1. **Estado inicial:** cláusulas unitárias em $t = 0$.
2. **Meta:** cláusulas unitárias em $t = T$.
3. **Unicidade de posição e nível:** $\bigvee_p at(b, p, t)$ e $\neg at(b, p_1, t) \vee \neg at(b, p_2, t)$; análogo para lev .
4. **Exclusão horizontal:** $\neg at(b_1, p_1, t) \vee \neg at(b_2, p_2, t) \vee \neg lev(b_1, l, t) \vee \neg lev(b_2, l, t)$ se os spans se sobrepõem.
5. **Estabilidade:** para cada combinação de posições dos outros blocos no nível $l - 1$ que dá menos de $\lceil \ell(b)/2 \rceil$ slots de apoio, a combinação é proibida.
6. **Clear:** $acima(x, y, t)$ vale sse x está no nível $lev(y) + 1$ com span sobreposto; $clr(y, t) \leftrightarrow \neg \bigvee_x acima(x, y, t)$.
7. **Pré-condições e efeitos de move:** cláusulas $\neg move(b, y, p, t) \vee$ (condição). A pré-condição (4) e o efeito sobre $clr(y)$ ficam garantidos pelos grupos 4, 5 e 6 no estado $t + 1$.
8. **Frame axioms:** $\neg at(b, p, t) \vee at(b, p, t + 1) \vee (b \text{ foi movido em } t)$; idem para lev .
9. **No máximo uma ação por passo.** Não exigimos “pelo menos uma”, então um plano de k ações vale para todo $T \geq k$.
10. **Ordem parcial (opcional).**

4 Exemplos dos Três Cenários Codificados

Formato: bloco : (p, l) (ponto inicial, nível).

Cenário	Estado inicial	Meta
Sit. 1 (S_{f4})	$c:(0,0) a:(3,0) b:(5,0) d:(3,1)$	$c:(0,0) a:(0,1) d:(2,0) b:(5,0)$
Sit. 1 (S_{f1})	idem	$d:(3,0) a:(4,1) b:(5,1) c:(4,2)$
Sit. 1 (S_{f2})	idem	$d:(3,0) c:(4,1) a:(4,2) b:(5,2)$
Sit. 1 (S_{f3})	idem	$c:(0,0) a:(2,0) d:(0,1) b:(5,0)$
Sit. 2 (S_5)	$c:(0,0) a:(0,1) b:(1,1) d:(3,0)$	$d:(3,0) c:(4,1) a:(4,2) b:(5,2)$
Sit. 3 (S_7)	$c:(0,0) a:(3,0) b:(5,0) d:(3,1)$	$c:(0,0) a:(0,1) b:(1,1) d:(3,0)$

Na Situação 3 a ordem parcial usada é $(a, 0, 1) \prec_P (b, 1, 1)$.

Estado inicial da Situação 1 (d em ponte sobre a e b ; o slot 4 está vazio sob d):

```
| . | . | . | d | d | d |
| c | c | . | a | . | b |
slots: 0 1 2 3 4 5
```

Em S_0 : $on(c, T)$, $on(a, T)$, $on(b, T)$, $on(d, a) \wedge on(d, b)$.

5 Mapeamento: Descrição Formal $\rightarrow$ Código

Regra formal	Código (bw2cnf_var.py)
Blocos e comprimentos	BLOCKS = {'a':1, 'b':1, 'c':2, 'd':3}
Pontos do eixo	MAX_POINT = 6
Posições válidas	valid_positions(L)
Slots cobertos	span(b, p)
Sobreposição de span	spans_overlap(b1,p1,b2,p2)
Apoio mínimo $\lceil \ell/2 \rceil$	min_apoio(b)
$at, lev, clr, move$	dicionários at, lev, clr, mv
Estado inicial / meta	blocos 3.1, 3.2
Unicidade	bloco 3.3 (A), (B)
Exclusão horizontal	bloco 3.3 (C)
Estabilidade	bloco 3.3 (D)
Clear	bloco 3.3 (E) (variável acima)
Pré-condições e efeitos de $move$	bloco 3.4 (G)
Frame axioms	bloco 3.5
Uma ação por passo	bloco 3.6
Ordem parcial	bloco 3.7 (campo order)
Relação on (derivada)	interpretar.py, derivar_on

6 Execução Passo a Passo

1. Instalar: `sudo apt install minisat` (Linux/WSL) ou `brew install minisat` (macOS).
2. Gerar o CNF e o mapa:

```
python3 bw2cnf_var.py situacao1 --horizon 4 --saida situacao1
```

Saída: Gerado: N variaveis, M clausulas, e os arquivos trab01_blocos2SAT.cnf e trab01_blocos2SAT.map.

3. Executar o miniSAT:

```
minisat situacao1/trab01_blocos2SAT.cnf situacao1/resultado1.txt
```

4. Interpretar:

```
python3 interpretar.py situacao1/resultado1.txt --verbose
```

5. Para todos os cenários, procurando o menor T : `python3 rodar.py`.

Resultados (menor T SATISFIABLE; todos os T menores deram UNSATISFIABLE):

Cenário	T mín.	Variáveis	Cláusulas
Sit. 1, $S_0 \rightarrow S_{f4}$	4	601	72179
Sit. 1, $S_0 \rightarrow S_{f1}$	8	1149	133299
Sit. 1, $S_0 \rightarrow S_{f2}$	9	1286	148579
Sit. 1, $S_0 \rightarrow S_{f3}$	2	327	41619
Sit. 2, $S_0 \rightarrow S_5$	5	738	87459
Sit. 3, $S_0 \rightarrow S_7$ ($a \prec b$)	6	882	102760
Sit. 3, $S_0 \rightarrow S_7$ (sem ordem)	6	875	102739
Sit. 3, $S_0 \rightarrow S_7$ ($b \prec a$)	7	1020	118043

A ordem $b \prec a$ alonga o plano em uma ação, o que mostra que a restrição de ordem parcial age de fato.

6.1 Planos encontrados

Cenário	Plano
Sit. 1 $\rightarrow S_{f4}$	<i>move</i> ($d, c, 0$); <i>move</i> ($a, b, 5$); <i>move</i> ($d, T, 2$); <i>move</i> ($a, c, 0$)
Sit. 1 $\rightarrow S_{f3}$	<i>move</i> ($d, c, 0$); <i>move</i> ($a, T, 2$)
Sit. 1 $\rightarrow S_{f1}$	<i>move</i> ($d, c, 0$); <i>move</i> ($a, T, 2$); <i>move</i> ($d, c, 1$); <i>move</i> ($b, c, 0$); <i>move</i> ($d, T, 3$); <i>move</i> ($b, d, 5$); <i>move</i> ($c, b, 4$); <i>move</i> ($a, d, 4$)
Sit. 1 $\rightarrow S_{f2}$	<i>move</i> ($c, T, 1$); <i>move</i> ($d, c, 0$); <i>move</i> ($a, T, 0$); <i>move</i> ($d, c, 1$); <i>move</i> ($b, a, 0$); <i>move</i> ($d, T, 3$); <i>move</i> ($c, d, 4$); <i>move</i> ($b, c, 5$); <i>move</i> ($a, c, 4$)
Sit. 2 $\rightarrow S_5$	<i>move</i> ($a, T, 2$); <i>move</i> ($b, a, 2$); <i>move</i> ($c, d, 4$); <i>move</i> ($b, c, 5$); <i>move</i> ($a, c, 4$)
Sit. 3 $\rightarrow S_7$ ($a \prec b$)	<i>move</i> ($d, c, 0$); <i>move</i> ($a, b, 5$); <i>move</i> ($d, T, 2$); <i>move</i> ($a, c, 0$); <i>move</i> ($b, c, 1$); <i>move</i> ($d, T, 3$)
Sit. 3 $\rightarrow S_7$ ($b \prec a$)	<i>move</i> ($d, c, 0$); <i>move</i> ($a, b, 5$); <i>move</i> ($d, T, 2$); <i>move</i> ($a, d, 3$); <i>move</i> ($b, c, 1$); <i>move</i> ($a, c, 0$); <i>move</i> ($d, T, 3$)

6.2 Execução manual: Situação 1, $S_0 \rightarrow S_{f4}$

```
t=0 |.|.|.|d|d|d| move(d,c,0): d sobre c, slots 0,1,2 livres;
    |c|c|.a|.b| apoio: c cobre 0,1 (2 >= ceil(3/2)=2)
t=1 |d|d|d|.|.|. move(a,b,5): a sobre b (a tem topo livre)
    |c|c|.a|.b|
t=2 |d|d|d|.|.a| move(d,T,2): d na mesa, slots 2,3,4 livres
    |c|c|.|.|.b|
t=3 |.|.|.|.|a| move(a,c,0): a sobre c, apoio 1 >= 1
    |c|c|d|d|b|
t=4 |a|.|.|.|. meta Sf4 atingida
    |c|c|d|d|b|
```

Por que a não vai para a mesa: depois do passo 1 os slots 0,1 têm c , o slot 5 tem b , e restam os slots 2,3,4, exatamente os que d ocupará na meta. Não há lugar livre na mesa, então a espera em cima de b . Os passos manuais das demais situações estão no `README.md`.

7 Interpretação da Saída do SAT Solver

7.1 O miniSAT não conhece nomes simbólicos

O miniSAT é puramente booleano e devolve uma lista de inteiros (os IDs das variáveis verdadeiras). A tradução inteiro $\rightarrow$ símbolo está no arquivo `trab01_blocos2SAT.map`, gerado por `bw2cnf_var.py`.

7.2 Da saída numérica ao plano em português

O `interpretar.py` (1) lê o mapa; (2) mantém os literais positivos que são ações *move*; (3) ordena por t e traduz cada ação em uma frase; e reconstrói os estados a partir de *at* e *lev*.

7.3 Derivação de *on*

Se $l_b = 0$: $on(b, T, t)$. Se $l_b > 0$: b está sobre todo y com $lev(y) = l_b - 1$ cujo span sobrepõe o de b . Exemplos: d está sobre a, b (ponte); a está sobre c .

7.4 Verificação independente

Com `-verbose` o script desenha cada estado e confere as regras físicas sem usar o CNF (sem sobreposição, estabilidade, topo livre, só o bloco movido muda). Todos os planos passaram.

7.5 Regra de ouro

Sem o arquivo `.map`, a saída do miniSAT é apenas uma lista de inteiros sem significado para o domínio. Nunca interprete a saída sem o mapa correspondente.